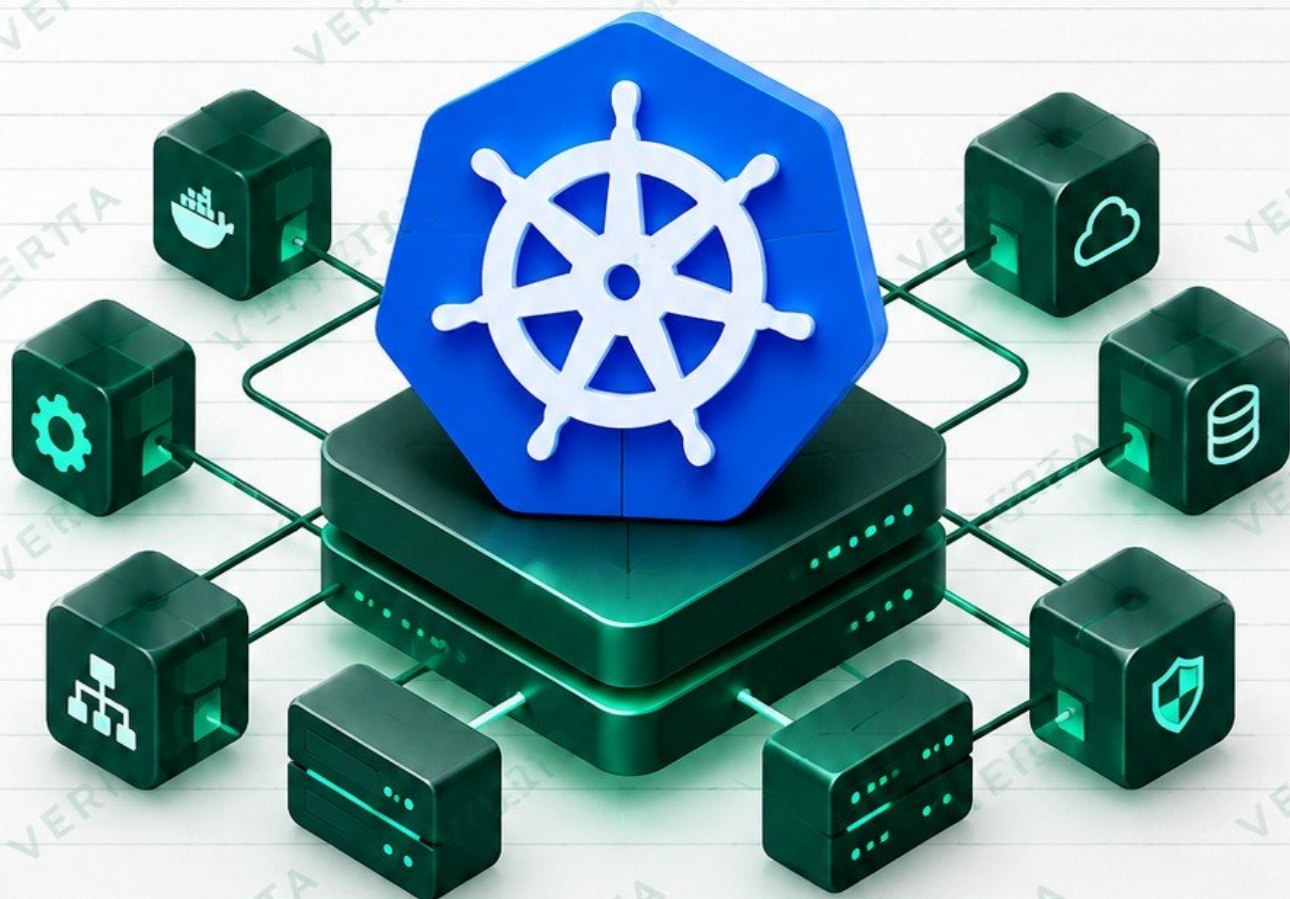


Kubernetes

Administrator's Survival Guide

**Common Cluster Problems
and Decisions**



01

The Kubernetes Administrator's Incident Mindset

Establish blast radius, identify recent changes, inspect cluster health, avoid destructive first actions, prioritize control plane and workload evidence.

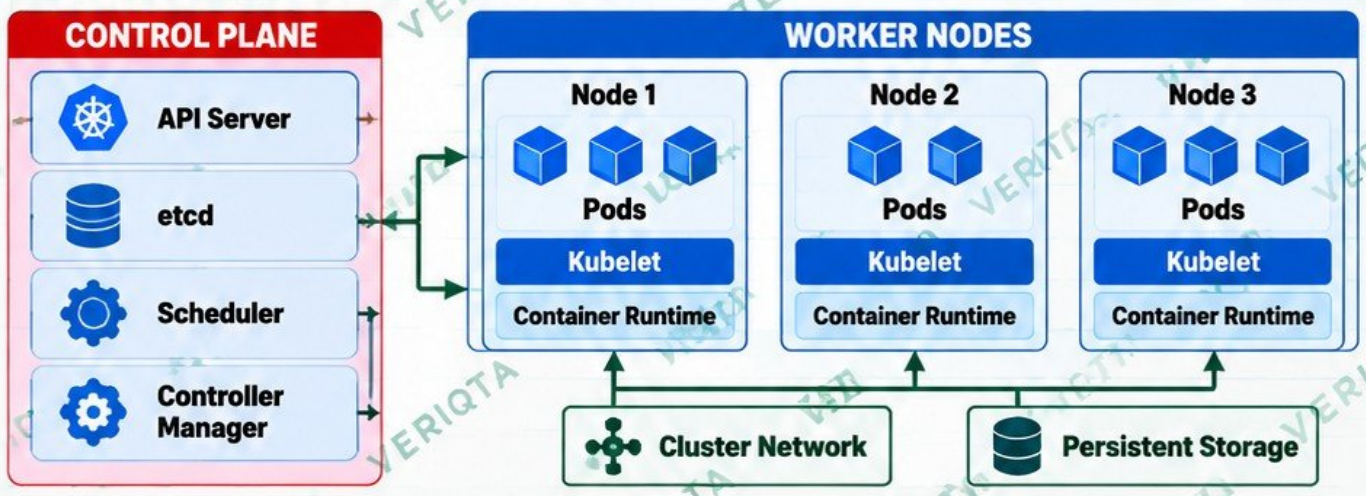
WHY THIS MATTERS

- Kubernetes failures can affect one Pod or the entire cluster.
- Quick, uncontrolled actions can make things worse and destroy evidence.
- A clear incident mindset helps you find the real cause, recover safely, and prevent recurrence.

THE INCIDENT MINDSET

- 1 Stay calm and assess the situation
- 2 Determine the blast radius
- 3 Identify recent changes
- 4 Inspect cluster health and key components
- 5 Gather evidence before making changes
- 6 Take the safest action to stabilize
- 7 Verify recovery and prevent recurrence

KUBERNETES CLUSTER OVERVIEW



SENIOR INCIDENT INVESTIGATION FLOW



USEFUL INVESTIGATION COMMANDS

<code>kubectl get nodes</code>	Check node status
<code>kubectl get pods -A</code>	Check all Pods
<code>kubectl get events -A</code>	Look for recent errors
<code>kubectl cluster-info</code>	Check control plane
<code>kubectl get componentstatuses</code>	Control plane health
<code>kubectl top nodes</code>	Node resource usage
<code>kubectl top pods -A</code>	Pod resource usage
<code>kubectl get svc -A</code>	Check services
<code>kubectl get ingress -A</code>	Check ingresses
<code>kubectl get pvc -A</code>	Check storage
<code>kubectl describe <resource></code>	Detailed information
<code>kubectl logs <pod> -c <container></code>	Application logs

AVOID THESE FIRST ACTIONS

- ❌ Do NOT restart Pods immediately.
- ❌ Do NOT delete resources without analysis.
- ❌ Do NOT change configurations during investigation.
- ❌ Do NOT scale down or drain nodes without checking impact.
- ❌ Do NOT modify the control plane unless absolutely necessary.

KEY TAKEAWAYS

- ✓ Understand the scope before taking action.
- ✓ Always collect evidence.
- ✓ Fix the root cause.
- ✓ Investigate first, then mitigate.
- ✓ Communicate impact and progress.
- ✓ Improve monitoring and alerting.

Scheduler events, resource requests, affinity, taints/tolerations, PVC binding, topology constraints, quotas, unschedulable nodes.

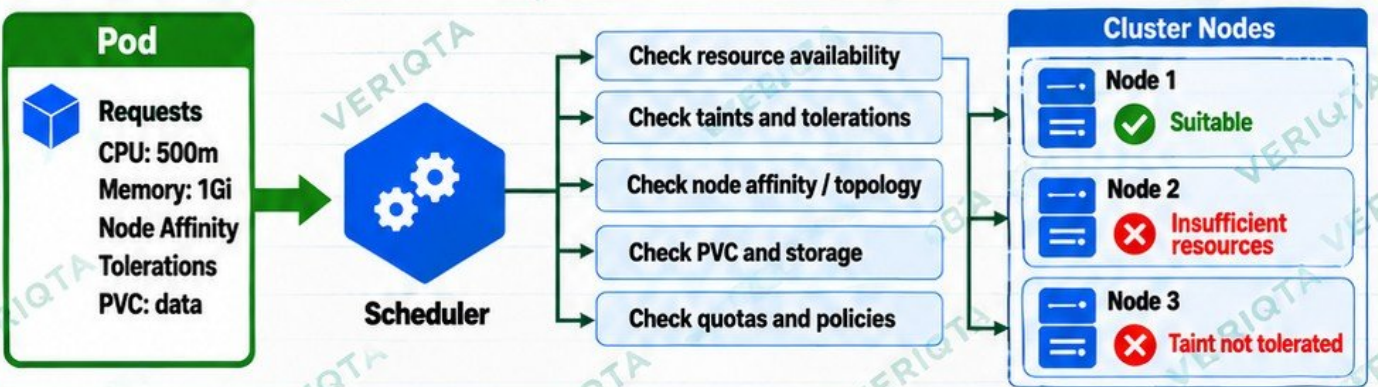
WHAT IT MEANS

- Pod is created but not scheduled to any node.
- The scheduler cannot find a suitable node that meets all requirements.
- Check events to understand why.

COMMON CAUSES

- 1 Insufficient CPU, memory or other resources
- 2 Node taints without matching tolerations
- 3 Node affinity or pod affinity rules
- 4 PersistentVolumeClaim not bound
- 5 Topology constraints (zones, node pools)
- 6 Resource quotas and limit ranges
- 7 All nodes unschedulable or NotReady

HOW THE SCHEDULER MAKES A DECISION



CHECK EVENTS FIRST

```
kubectl get pods -A
kubectl describe pod <pod-name>
kubectl get events -A --sort-by=.lastTimestamp
```

Look for messages like:

- "Insufficient cpu"
- "Insufficient memory"
- "node(s) had taint ..."
- "persistentvolumeclaim not bound"
- "node(s) didn't match node affinity"
- "exceeded quota"
- "node(s) were unschedulable"

OTHER USEFUL CHECKS

```
kubectl get nodes -o wide
Check node capacity and status

kubectl describe node <node>
Check taints and conditions

kubectl get pvc -A
Check if PVC is bound

kubectl get storageclass
Check storage class and provisioner

kubectl get quota -A
Check resource quotas

kubectl get limitrange -A
Check default resource limits

kubectl get pods -A -o wide
Check node distribution
```

EXAMPLE EVENT OUTPUT

Events:	Type	Reason	Message
Warning	FailedScheduling	0/3 nodes are available:	
		1 Insufficient cpu,	
		1 Insufficient memory,	
		1 node(s) had taint	
		node-role.kubernetes.io/	
		gpu:NoSchedule.	

POSSIBLE FIXES

- ✓ Increase cluster node capacity
- ✓ Adjust resource requests or limits
- ✓ Add required tolerations
- ✓ Fix node affinity or topology rules
- ✓ Bind or create the required PVC
- ✓ Check and adjust quotas/limits
- ✓ Uncordon or fix unschedulable nodes

SENIOR ENGINEER NOTES

- ✓ Do not just remove resource requests. Understand actual requirements.
- ✓ Pending Pods can indicate a deeper capacity or configuration issue.
- ✓ Check recent changes: new quotas, taints, topology rules, or node pool changes.
- ✓ For stateful workloads, confirm storage and topology alignment.
- ✓ Use cluster autoscaler or add nodes if this is a genuine capacity problem.

COMMON DECISIONS

- 1 Is this a temporary capacity issue or configuration issue?
- 2 Should I add nodes, adjust requests, or change placement rules?
- 3 Do I need to modify taints/tolerations or storage configuration?
- 4 Is this a one-off issue or a policy-wide problem (quota/limit)?
- 5 Will this affect other workloads or just this namespace?

Previous container logs, exit codes, OOM, failed startup, dependency failures, bad config, backoff behavior, when restart hides evidence.

WHAT IT MEANS

- The container starts, fails, and restarts repeatedly.
- Kubelet increases the restart delay (backoff).
- Usually caused by application errors, misconfiguration, resource limits, or dependency failures.
- Restarting without investigation can hide the real problem.

HOW BACKOFF WORKS



COMMON CAUSES

- 1 Application crashes (bugs, bad config)
- 2 Exit with non-zero code
- 3 Out of memory (OOMKilled)
- 4 Missing or wrong environment variables
- 5 Dependency not ready (database, API, etc.)
- 6 Invalid arguments or command
- 7 Permission issues (filesystem, secrets)
- 8 Probe failures (liveness/startup)

TROUBLESHOOTING STEPS

- 1 Check the pod status and restart count
- 2 Inspect previous container logs
- 3 Check the last exit code and reason
- 4 Look for events and probe failures
- 5 Verify configuration, env vars and secrets
- 6 Check resource usage and OOM issues
- 7 Validate dependencies (DB, services, DNS)
- 8 Fix the root cause and redeploy

USEFUL COMMANDS

<code>kubectl get pods -A</code>	Check pod status and restarts
<code>kubectl describe pod <pod></code>	View events and failure reasons
<code>kubectl logs <pod> -c <container></code>	View current logs
<code>kubectl logs <pod> -c <container> --previous</code>	View previous container logs
<code>kubectl get events -A --sort-by=.lastTimestamp</code>	Check recent events
<code>kubectl top pod <pod></code>	Check resource usage

READING POD STATUS

STATUS / REASON	MEANING	WHAT TO DO
CrashLoopBackOff	Container keeps failing and restarting	Check logs and root cause
Error	Container exited with error	Check previous logs and exit code
OOMKilled	Killed due to out of memory	Check memory limits and usage
Completed	Container finished (success or failed)	Check exit code and job design
RunContainerError	Failed to start container	Check command, image, permissions
CreateContainerError	Error creating container	Check config, secrets, volumes, image

EXAMPLE: APPLICATION CRASH

```

$ kubectl logs my-app-7d9f5c6b7f-abcde --previous
Starting application...
Error: missing environment variable DATABASE_URL
exit status 1
  
```

KEY TAKEAWAYS

- ✓ Do not keep restarting. Find the root cause.
- ✓ Always check previous logs and exit codes.
- ✓ Look beyond the application (resources, dependencies).
- ✓ Fix the underlying issue instead of increasing restart limits.
- ✓ Use probes correctly to avoid unnecessary restarts.



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

Registry reachability, image names/tags, authentication, imagePullSecrets, ServiceAccounts, rate limits, DNS/TLS failures, supply-chain considerations.

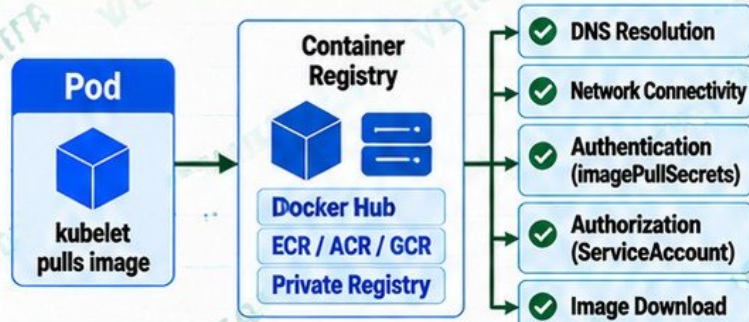
WHAT IT MEANS

- Kubernetes cannot pull the container image from the registry.
- ImagePullBackOff means repeated failed pulls with increasing backoff.
- ErrImagePull is the initial error when the image pull fails.
- The Pod stays in Pending or restarts without the image.

COMMON CAUSES

- 1 Wrong image name or tag
- 2 Registry not reachable (network, DNS, firewall)
- 3 Authentication failure (missing or invalid credentials)
- 4 Missing or incorrect imagePullSecrets
- 5 ServiceAccount does not have access
- 6 Rate limiting from the registry
- 7 TLS or certificate errors
- 8 Private registry permissions
- 9 Image does not exist
- 10 Supply-chain or image scanning policy blocking

HOW IMAGE PULL WORKS



CHECK THESE FIRST

- 1 Check Pod events and describe
- 2 Verify image name and tag
- 3 Test registry connectivity from node
- 4 Check DNS resolution
- 5 Check authentication credentials
- 6 Verify imagePullSecrets
- 7 Check ServiceAccount permissions
- 8 Look for registry rate limits
- 9 Check TLS/certificate issues
- 10 Verify the image exists and is accessible

USEFUL COMMANDS

<code>kubectl get pods -A</code>	Check Pod status
<code>kubectl describe pod <pod></code>	View events and pull errors
<code>kubectl get events -A \\ --sort-by=.lastTimestamp</code>	Check recent events
<code>kubectl logs <pod> -c <container></code>	View container logs (if available)
<code>kubectl get sa <sa> -o yaml</code>	Check ServiceAccount
<code>kubectl get secret <secret> -o yaml</code>	Check imagePullSecret
<code>kubectl get nodes -o wide</code>	Check node network and status
<code>nslookup docker.io</code>	Test DNS resolution
<code>curl -v https://index.docker.io/v2/</code>	Test registry connectivity

COMMON EVENT MESSAGES

MESSAGE	LIKELY CAUSE
ErrImagePull	Initial image pull failure
ImagePullBackOff	Repeated failures (backoff)
manifest unknown	Image or tag does not exist
unauthorized	Authentication failure
denied: requested access to the resource	Permission or policy issue
x509: certificate unknown authority	TLS/certificate problem
dial tcp: i/o timeout	Network, DNS or firewall issue
too many requests	Registry rate limiting

EXAMPLE: POD EVENT OUTPUT

```
$ kubectl describe pod my-app
Events:
  Type     Reason      Message
  ---     -
  Warning   Failed      Failed to pull image "myrepo/app:1.0"
  Warning   Failed      Error: ImagePullBackOff
  Normal   BackOff     Back-off pulling image "myrepo/app:1.0"
```

FIXES AND BEST PRACTICES

- ✓ Use correct image name and tag
- ✓ Configure imagePullSecrets for private registries
- ✓ Ensure ServiceAccount has read access
- ✓ Verify DNS, network and firewall allow registry access
- ✓ Handle registry rate limits (caching, mirrors)
- ✓ Use trusted and scanned images
- ✓ Pin image digests for production workloads

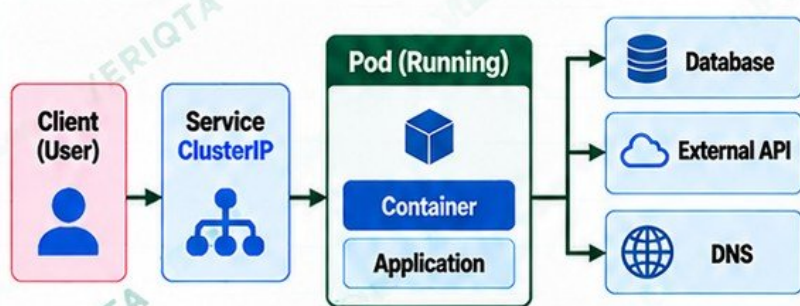
THE PROBLEM

- Pod shows Running but the application is not working.
- Users cannot access the service or get errors.
- The container process may be healthy but the application is failing.
- Readiness probe, configuration, secrets, ports or dependencies can cause this.

COMMON CAUSES

- Application started but not ready
- Readiness probe misconfigured
- Wrong container port or service port
- Missing or incorrect configuration
- Secrets or environment variables missing
- Database or upstream dependency down
- DNS resolution issues
- NetworkPolicy or firewall blocking traffic
- Application errors in logs
- Service or Ingress misconfiguration

HOW TRAFFIC FLOWS



POD CAN BE RUNNING BUT STILL BROKEN

Component	Possible Issue
Container process	Process is running but app not ready
Readiness probe	Misconfigured or not checking real app
Service	Wrong selector or targetPort
Configuration	Missing or incorrect config values
Secrets	Wrong or missing secrets
Dependencies	Database, API or cache not reachable
DNS	Cannot resolve service or external name
Network	Blocked by NetworkPolicy or firewall
Application logs	Application startup or runtime errors

INVESTIGATION STEPS

- Check Pod, container and readiness status
- Check recent events and logs
- Verify container ports and service configuration
- Inspect application logs inside the container
- Check configuration, environment variables and secrets
- Test connectivity to dependencies (DNS, DB, APIs)
- Check NetworkPolicy, security groups or firewall rules
- Verify Ingress or external load balancer if applicable
- Identify the real error and fix the root cause

USEFUL COMMANDS

<code>kubectl get pods -A</code>	Check Pod status
<code>kubectl describe pod <pod></code>	View events and probe status
<code>kubectl get events -A --sort-by=.lastTimestamp</code>	Check recent events
<code>kubectl logs <pod> -c <container></code>	View application logs
<code>kubectl get svc -A</code>	Check service and endpoints
<code>kubectl get endpoints <svc></code>	Verify target Pods
<code>kubectl exec -it <pod> --- sh</code>	Test connectivity from Pod
<code>kubectl get configmap,secret -A</code>	Check configuration and secrets
<code>kubectl get ingress -A</code>	Check ingress configuration
<code>kubectl get networkpolicy -A</code>	Check network policies
<code>nslookup <service></code>	Test DNS resolution
<code>curl -v <service>:<port></code>	Test service connectivity

EXAMPLE: RUNNING BUT UNHEALTHY APP

```

$ kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
my-app-7d8f  1/1     Running   0          2h

$ kubectl logs my-app-7d8f
INFO    Starting application...
ERROR   Cannot connect to database: db:5432
Caused by: java.net.ConnectException: Connection refused
  
```

KEY TAKEAWAYS

- Pod Running does not mean the application is healthy.
- Always check readiness probe and actual application health.
- Inspect logs, events and configuration.
- Verify service, ports and network connectivity.
- Check dependencies such as databases, APIs and DNS.
- Fix the real root cause instead of just restarting the Pod.
- Use proper monitoring and alerting to detect application failures.

Service selectors, EndpointSlices, targetPort, readiness, kube-proxy/service routing, NetworkPolicy, connection testing and traffic-path isolation.

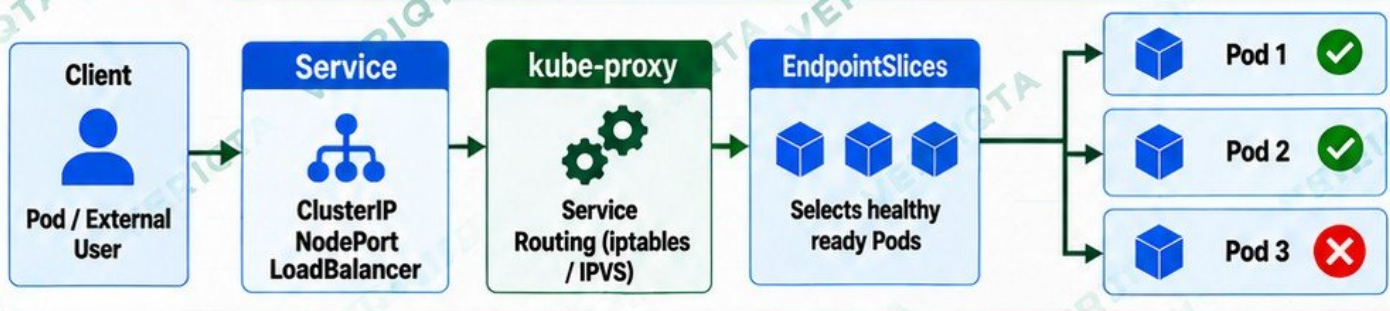
WHAT IT MEANS

- The Service is created, but traffic cannot reach the application.
- Endpoints may be missing or not ready.
- Traffic can be blocked by NetworkPolicy, firewall or misconfiguration.
- The issue can be within the cluster or from external clients.

COMMON CAUSES

- 1 Incorrect or non-matching Service selector
- 2 No ready endpoints (pods not ready)
- 3 Wrong targetPort or port configuration
- 4 Pod readiness probe failing
- 5 kube-proxy or CNI networking issue
- 6 NetworkPolicy blocking traffic
- 7 DNS resolution issues
- 8 Firewall or cloud security group blocking traffic
- 9 Application not listening on the expected port
- 10 Ingress or load balancer misconfiguration (for external access)

HOW A SERVICE ROUTES TRAFFIC



INVESTIGATION STEPS

- 1 Verify the Service and its configuration
- 2 Check if selectors match the Pod labels
- 3 Confirm EndpointSlices and ready endpoints
- 4 Check Pod readiness and container health
- 5 Verify targetPort and container port
- 6 Test internal connectivity from another Pod
- 7 Check DNS resolution for the Service
- 8 Review NetworkPolicy and firewall rules
- 9 Test external access (Ingress/LoadBalancer)
- 10 Isolate where the traffic is failing

USEFUL COMMANDS

<code>kubectl get svc -A</code>	List all Services
<code>kubectl describe svc <svc></code>	View Service details
<code>kubectl get endpointslices -l</code> <code>kubernetes.io/service-name=<svc></code>	Check EndpointSlices
<code>kubectl get endpoints <svc></code>	Check endpoints (legacy)
<code>kubectl get pods -l <selector></code>	Check matching Pods
<code>kubectl get pods -o wide</code>	Check Pod IPs and nodes
<code>kubectl describe pod <pod></code>	Check readiness and events
<code>kubectl logs <pod></code>	View application logs
<code>kubectl exec -it <pod> -- ...</code>	Test connectivity (curl, nc)
<code>kubectl get networkpolicy -A</code>	Check NetworkPolicies
<code>nslookup <svc></code>	Test DNS resolution
<code>curl -v http://<svc>:<port></code>	Test service connectivity

CHECK SERVICE SELECTOR

Service selector must match Pod labels:

```

apiVersion: v1
kind: Service
metadata:
  name: my-service
spec:
  selector:
    app: my-app # must match Pod labels
  ports:
    - port: 80
      targetPort: 8080
  
```

CHECK ENDPOINTSlices

```

kubectl get endpointslices -l
kubernetes.io/service-name=my-service
  
```

NAME	ENDPOINTS	READY
my-svc-abc	10.244.1.5	true
my-svc-def	10.244.2.6	true

TEST CONNECTIVITY FROM ANOTHER POD

```

# Start a temporary pod
kubectl run -it test --image=busybox --rm -- sh

# Test DNS resolution
nslookup my-service

# Test service connectivity
curl -v http://my-service:80

# Test specific port
nc -zv my-service 8080
  
```

COMMON ISSUES AND SIGNS

SYMPTOM	LIKELY CAUSE
No endpoints	Selector mismatch or Pods not ready
Endpoints not ready	Readiness probe failing
Connection refused	Wrong targetPort or app not listening
Timeout	NetworkPolicy, firewall or routing issue
DNS not resolving	CoreDNS or DNS configuration issue
Works inside cluster but not external	Ingress/LoadBalancer or firewall issue
Intermittent failures	Unready Pods or network instability

KEY TAKEAWAYS

- ✓ A Service only routes to ready endpoints.
- ✓ Always verify selectors, endpoints and readiness probes.
- ✓ Check targetPort and container ports.
- ✓ Use EndpointSlices for accurate endpoint info.
- ✓ Test connectivity step by step (DNS, port, app).
- ✓ Review NetworkPolicy and firewall rules.
- ✓ Isolate the traffic path to find where it fails.
- ✓ Do not assume the Service is the problem.

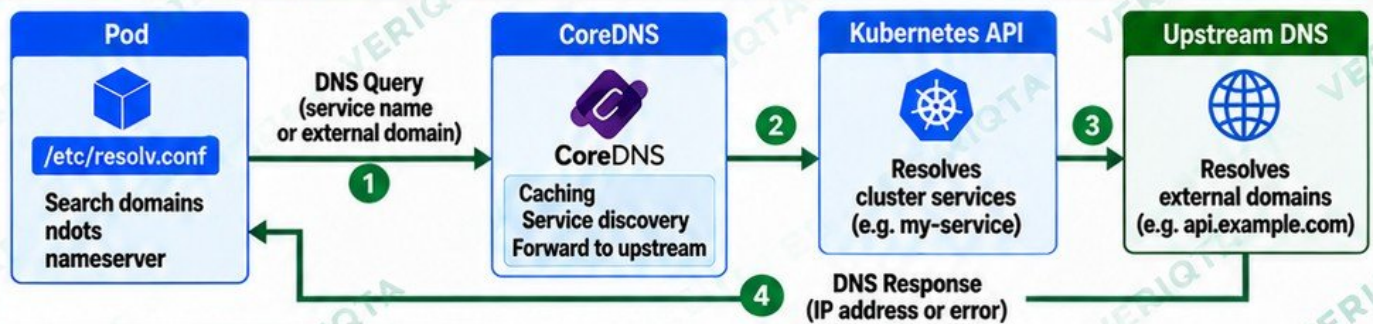
WHAT IT MEANS

- Pods cannot resolve service names or external domains.
- DNS queries timeout, return errors or give wrong results.
- Service discovery inside the cluster is not working.
- The issue can be with CoreDNS, configuration, upstream DNS or network connectivity.

COMMON CAUSES

- 1 CoreDNS pods not running or unhealthy
- 2 Wrong or missing CoreDNS configuration
- 3 Upstream DNS server issues
- 4 Incorrect /etc/resolv.conf settings
- 5 Search domain or ndots misconfiguration
- 6 NetworkPolicy or firewall blocking DNS traffic
- 7 DNS query saturation (high load)
- 8 Caching issues or stale records
- 9 Service or external domain does not exist
- 10 Node DNS resolution problems

HOW KUBERNETES DNS WORKS



INVESTIGATION STEPS

- 1 Check if CoreDNS pods are running and healthy
- 2 Test DNS resolution from inside a pod
- 3 Check /etc/resolv.conf configuration
- 4 Verify CoreDNS configuration and logs
- 5 Check upstream DNS connectivity
- 6 Look for NetworkPolicy or firewall issues
- 7 Validate search domains and ndots settings
- 8 Check for high DNS query load or timeouts
- 9 Capture packets if needed
- 10 Identify the real cause and fix it

USEFUL COMMANDS

<code>kubectl get pods -n kube-system</code>	Check CoreDNS status
<code>kubectl logs -n kube-system \</code> <code>-l k8s-app=kube-dns</code>	View CoreDNS logs
<code>kubectl describe configmap</code> <code>coredns -n kube-system</code>	Check CoreDNS config
<code>kubectl get svc -n kube-system</code>	Verify CoreDNS service
<code>kubectl exec -it <pod> -- nslookup</code> <code>my-service</code>	Test service discovery
<code>kubectl exec -it <pod> -- nslookup</code> <code>api.example.com</code>	Test external DNS
<code>kubectl exec -it <pod> -- cat</code> <code>/etc/resolv.conf</code>	View DNS configuration
<code>kubectl get networkpolicy -A</code>	Check NetworkPolicies
<code>kubectl get endpoints <service></code>	Verify service endpoints

CHECK /etc/resolv.conf

```
$ cat /etc/resolv.conf
nameserver 10.96.0.10
search default.svc.cluster.local
svc.cluster.local cluster.local
options ndots:5
```

IMPORTANT FIELDS

- ✓ nameserver: IP of CoreDNS
- ✓ search: domains to try
- ✓ ndots: number of dots before absolute query (default 5)
- ✓ options: other resolver options

TEST FROM A POD

```
$ kubectl run -it --rm dns-test
--image=busybox -- sh
/ # nslookup my-service
/ # nslookup kubernetes.default
/ # nslookup api.example.com
/ # dig my-service.svc.cluster.local
```

COMMON DNS ERRORS

ERROR MESSAGE	LIKELY CAUSE
no such host	Service or domain not found
connection timed out	CoreDNS or upstream DNS issue
server misbehaving	CoreDNS overloaded or failing
NXDOMAIN	Domain does not exist
i/o timeout	Network, firewall or upstream DNS
temporary failure in name resolution	General DNS failure

FIXES AND BEST PRACTICES

- ✓ Ensure CoreDNS pods are healthy and scaled.
- ✓ Verify correct CoreDNS configuration.
- ✓ Check upstream DNS servers and connectivity.
- ✓ Configure appropriate search domains and ndots.
- ✓ Allow DNS traffic in NetworkPolicy and firewalls.
- ✓ Monitor DNS query load and latency.
- ✓ Use caching and proper TTL settings.
- ✓ Validate service names, namespaces and domains.

KEY TAKEAWAYS

- ✓ Kubernetes DNS is critical for service discovery and external access.
- ✓ Always check CoreDNS pods and logs first.
- ✓ Validate /etc/resolv.conf, search domains and ndots.
- ✓ Test DNS from inside a pod.
- ✓ Check NetworkPolicy and upstream DNS.
- ✓ Use proper monitoring to detect DNS issues.
- ✓ Do not assume the application or service is the problem.

Node NotReady: Drain, Repair, or Replace?

Node conditions, kubelet, runtime, network, disk/memory/PID pressure, heartbeats, workloads at risk, cordon/drain decisions.

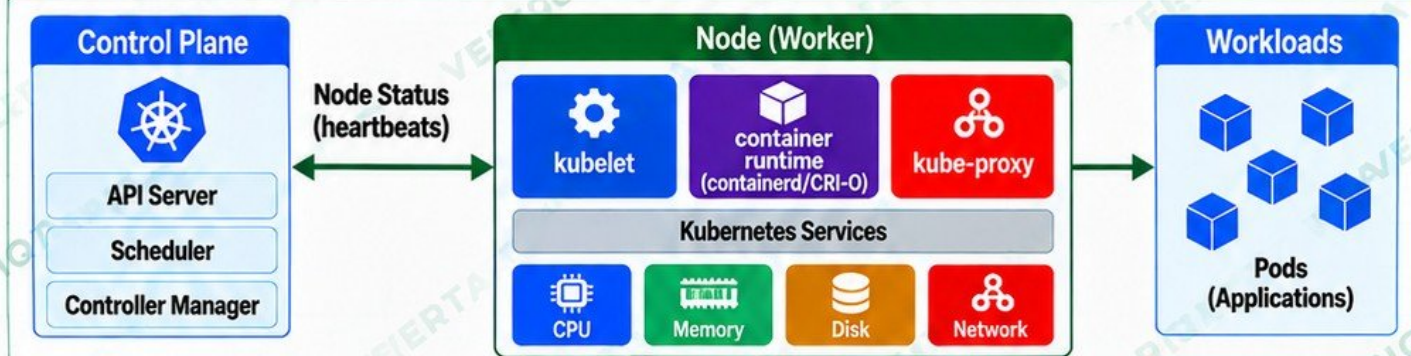
WHAT IT MEANS

- Node shows NotReady and cannot accept new workloads.
- Kubelet is not reporting healthy status to the control plane.
- Existing Pods on the node may be at risk.
- Caused by kubelet, runtime, network, resource pressure or underlying OS issues.

COMMON CAUSES

- 1 Kubelet not running or crashed
- 2 Container runtime issues (containerd/CRI-O)
- 3 Network problems (node cannot reach API server)
- 4 Disk pressure (node disk full)
- 5 Memory pressure (insufficient memory)
- 6 PID pressure (too many processes)
- 7 Node conditions failing (Ready, MemoryPressure, etc.)
- 8 Heartbeat/lease not renewed
- 9 Underlying OS or kernel issues
- 10 Hardware failure or misconfiguration

KUBERNETES NODE ARCHITECTURE (KEY COMPONENTS)



INVESTIGATION STEPS

- 1 Check node status and conditions
- 2 Describe the node to see failure reasons
- 3 Check kubelet status and logs
- 4 Check container runtime status
- 5 Check network connectivity to API server
- 6 Check disk, memory and PID pressure
- 7 Identify Pods running on the node
- 8 Decide to cordon, drain, repair or replace
- 9 Verify node recovery and workloads
- 10 Investigate underlying OS or hardware issues

USEFUL COMMANDS

<code>kubectl get nodes</code>	List all nodes and status
<code>kubectl describe node <node></code>	View node conditions and events
<code>kubectl get pods -o wide</code>	See pods on each node
<code>kubectl cordon <node></code>	Mark node unschedulable
<code>kubectl drain <node></code>	Evict pods safely
<code>kubectl uncordon <node></code>	Allow scheduling again
<code>systemctl status kubelet</code>	Check kubelet status (on node)
<code>journalctl -u kubelet -f</code>	View kubelet logs (on node)
<code>systemctl status containerd</code>	Check container runtime
<code>df -h</code>	Check disk usage
<code>free -h</code>	Check memory usage
<code>ps aux wc -l</code>	Check number of processes
<code>ping <api-server-ip></code>	Test network connectivity
<code>curl -k https://<api-server-ip>:6443</code>	Test API server reachability

CHECK NODE CONDITIONS

<code>kubectl describe node <node></code>			
Conditions:			
Type	Status	Reason	
Ready	False	KubeletNotReady	
MemoryPressure	False	KubeletHasSufficientMemory	
DiskPressure	True	KubeletHasDiskPressure	
PIDPressure	False	KubeletHasSufficientPID	
NetworkUnavailable	False	RouteCreated	

CHECK RESOURCE USAGE (ON NODE)

```
# Disk usage
df -h
# Memory usage
free -h
# Number of processes
ps aux | wc -l
```

CORDON / DRAIN DECISIONS

ACTION	WHEN TO USE
Cordon	Stop new pods from scheduling for investigation or maintenance
Drain	Evict existing pods safely before repair or replacement
Repair	Fix kubelet, runtime, network, OS or resource issues
Replace	Use a new node if hardware or OS is failed

COMMON SIGNS AND SYMPTOMS

SYMPTOM	LIKELY CAUSE
Node NotReady	Kubelet down or network issue
Disk pressure	Disk full or high inode usage
Memory pressure	Insufficient memory or memory leak
PID pressure	Too many processes
Pods evicted	Node resource pressure
Node flapping	Network or kubelet instability
Cannot join cluster	TLS, network or configuration issue

KEY TAKEAWAYS

- Always check node conditions and events first.
- Verify kubelet, runtime and network connectivity.
- Check disk, memory and PID pressure.
- Identify workloads at risk before taking action.
- Use cordon and drain for safe maintenance.
- Repair or replace based on the root cause.
- Monitor nodes to detect issues early.
- Keep nodes healthy and up to date.



WHAT IT MEANS

- Pods are killed or evicted because of high memory usage.
- OOMKilled means the container was killed by the kernel due to out of memory.
- Evictions happen when the node is under memory pressure.
- Usually caused by wrong resource requests/limits, memory leaks or insufficient node memory.

COMMON CAUSES

- Container memory usage exceeds its limit
- Memory leak in the application
- Wrong or missing resource requests and limits
- Node memory pressure (not enough free memory)
- Other Pods consuming too much memory
- Kernel OOM killer terminates the process
- Kubelet evicts Pods due to memory pressure
- Large in-memory caches or buffers
- Unexpected traffic or workload spikes
- Insufficient node size or poor right-sizing

HOW MEMORY WORKS IN KUBERNETES



INVESTIGATION STEPS

- Check Pod status, events and exit codes
- Check container memory usage and limits
- Review node memory pressure and conditions
- Inspect application logs for memory errors
- Identify if it is OOMKilled or kubelet eviction
- Check recent changes (deployments, config, traffic)
- Verify QoS class and resource configuration
- Look for memory leaks or large cache usage
- Check other Pods on the node
- Decide on fix and right-sizing

USEFUL COMMANDS

<code>kubectl get pods -A</code>	List all Pods
<code>kubectl describe pod <pod></code>	Check events and reason
<code>kubectl logs <pod></code>	View application logs
<code>kubectl top pod <pod></code>	Check container memory usage
<code>kubectl top node</code>	Check node memory usage
<code>kubectl describe node <node></code>	Check node conditions
<code>kubectl get events --sort-by=.lastTimestamp</code>	View recent events
<code>kubectl get pod <pod> -o yaml</code>	Check resource requests/limits
<code>kubectl get node <node> -o yaml</code>	Check node allocatable and capacity
<code>free -h</code>	Check memory on node (SSH)
<code>dmesg grep -i oom</code>	Check kernel OOM logs (SSH)
<code>journalctl -u kubelet -f</code>	Check kubelet eviction logs (SSH)

POD STATUS EXAMPLES

```
$ kubectl get pods
```

NAME	STATUS	RESTARTS	AGE
api-7d8f9	OOMKilled	5	10m
worker-5f6d	Evicted	0	3m
web-6c7d8	Running	0	1h

CHECK EXIT CODE

```
$ kubectl describe pod api-7d8f9
```

Last State:	Terminated
Reason:	OOMKilled
Exit Code:	137
Started:	Fri, 26 Sep 2026 10:00:00
Finished:	Fri, 26 Sep 2026 10:01:00

NODE MEMORY PRESSURE

```
$ kubectl describe node worker-1
```

Conditions:

Type	Status	Reason	Message
MemoryPressure	True	KubeletHasInsufficientMemory	node memory is low

```
$ kubectl top node
```

NAME	CPU(cores)	MEMORY(%)	MEMORY(bytes)
worker-1	0.5	95%	30Gi
worker-2	0.2	45%	14Gi

COMMON SIGNS AND EVENTS

EVENT MESSAGE	MEANING
OOMKilled	Kernel killed process (exit 137)
Evicted	Kubelet evicted the Pod
MemoryPressure	Node memory is low
FailedAlloc	Not enough memory to schedule
Back-off restarting	Container keeps restarting
Container killed due to memory usage	Exceeded memory limit
Node has memory pressure	System under memory pressure

FIXES AND BEST PRACTICES

- Set appropriate resource requests and limits
- Right-size based on real usage (working set)
- Fix memory leaks in the application
- Use QoS Guaranteed for critical workloads
- Monitor node memory and set alerts
- Use Horizontal Pod Autoscaler (HPA) if needed
- Optimize in-memory caches and configurations
- Use larger nodes or more nodes if required
- Review and tune eviction thresholds
- Regularly analyze memory usage and trends

CPU Throttling and the "Healthy but Slow" Cluster

CPU limits, CFS throttling, requests, contention, latency, saturation, HPA signals, performance evidence and capacity decisions.

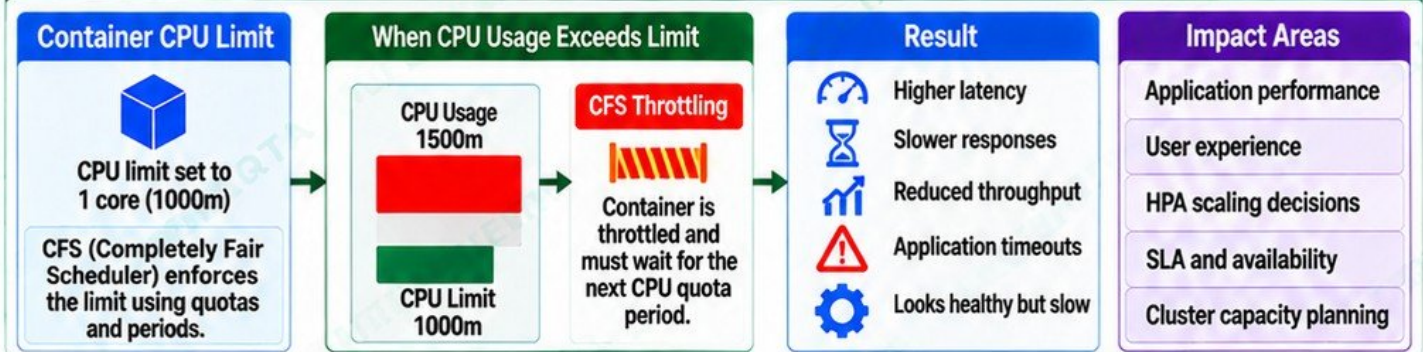
WHAT IT MEANS

- Pods are running and look healthy but the application is slow.
- CPU throttling happens when a container reaches its CPU limit (CFS throttling).
- The cluster may have enough CPU but poor request/limit settings, contention or saturation can cause slow performance.
- It leads to high latency, timeouts and reduced throughput.

COMMON CAUSES

- 1 CPU limit is too low for the workload
- 2 CFS throttling due to hitting CPU limit
- 3 Incorrect or missing CPU requests
- 4 High CPU contention on the node
- 5 Application CPU saturation
- 6 Too many Pods on the same node
- 7 Inefficient application code or heavy processing
- 8 HPA not scaling (wrong metrics or thresholds)
- 9 Background jobs or noisy neighbours
- 10 Insufficient cluster capacity or right-sizing

HOW CPU THROTTLING WORKS



INVESTIGATION STEPS

- 1 Check Pod CPU usage, requests and limits
- 2 Identify CPU throttling from metrics
- 3 Review node CPU utilization and contention
- 4 Check application performance and latency
- 5 Verify HPA metrics and scaling behavior
- 6 Look for noisy neighbours or high CPU Pods
- 7 Analyze recent changes or deployments
- 8 Check if the application is CPU bound
- 9 Review cluster capacity and right-sizing
- 10 Take action (increase limits, optimize app, scale)

USEFUL COMMANDS

<code>kubectl get pods -A</code>	List all Pods
<code>kubectl top pods</code>	View Pod CPU usage
<code>kubectl top nodes</code>	View node CPU usage
<code>kubectl describe pod <pod></code>	Check CPU requests/limits
<code>kubectl get hpa</code>	View HPA status
<code>kubectl describe hpa <name></code>	Check HPA configuration
<code>kubectl logs <pod></code>	View application logs
<code>kubectl exec -it <pod> -- top</code>	Check CPU inside container
<code>kubectl exec -it <pod> -- cat /sys/fs/cgroup/cpu/cpu.stat</code>	View CFS throttling stats
<code>kubectl get events --sort-by=.lastTimestamp</code>	Check recent events
<code>kubectl get nodes -o wide</code>	Check node CPU allocatable

CHECK CPU THROTTLING

```
$ kubectl exec -it my-pod -- cat /sys/fs/cgroup/cpu/cpu.stat
nr_periods      102400
nr_throttled    45678
throttled_time  1234567890
```

CHECK METRICS (PROMETHEUS)

```
# CPU throttling rate
rate(container_cpu_cfs_throttled_seconds_total[5m])

# CPU usage vs limit
container_cpu_usage_seconds_total
```

EXAMPLE: POD METRICS (kubectl top)

```
$ kubectl top pods
NAME          CPU(cores)  MEMORY(bytes)
api-7d8f9     1.2         200Mi
worker-5fd6   0.9         180Mi
web-6c7d8     0.1         90Mi
```

The api Pod is using 1.2 cores but may have a limit of 1 core, which can cause throttling.

COMMON SIGNS AND METRICS

SIGN	INDICATES
High latency	CPU throttling or saturation
Slow responses	Insufficient CPU or contention
High throttling rate	Hitting CPU limits (CFS throttling)
HPA not scaling	Wrong metrics or thresholds
CPU usage near 100%	CPU saturation
Frequent timeouts	Application cannot get CPU time
Node high CPU usage	Contention or insufficient capacity
Poor throughput	CPU or application bottleneck

KEY TAKEAWAYS

- Set realistic CPU requests and limits.
- Monitor CPU throttling and latency.
- Use HPA with the right metrics and thresholds.
- Check node CPU utilization and contention.
- Identify and fix noisy neighbours.
- Optimize application code and workloads.
- Use performance metrics to make capacity decisions.
- A healthy Pod does not always mean a healthy application.

DiskPressure, Ephemeral Storage and Full Nodes

Image layers, container logs, emptyDir, ephemeral-storage requests/limits, inode exhaustion, garbage collection, safe cleanup.

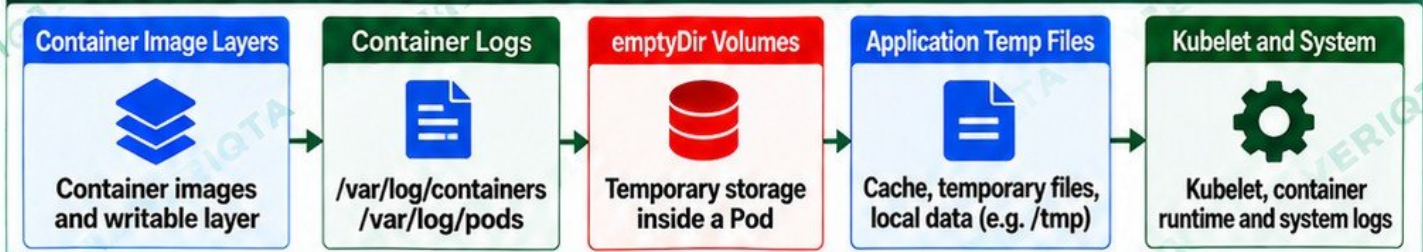
WHAT IT MEANS

- The node is running low on disk space or inodes and shows DiskPressure.
- Pods may be evicted or fail to start due to insufficient ephemeral storage.
- Caused by large container logs, image layers, emptyDir volumes, temporary files or inode exhaustion.
- If not resolved, it can lead to application outages and node instability.

COMMON CAUSES

- 1 Large container logs
- 2 Too many or large container images (image layers)
- 3 Applications writing a lot of temporary files
- 4 emptyDir volumes filling up
- 5 Missing or incorrect ephemeral-storage requests/limits
- 6 Inode exhaustion (too many files)
- 7 Old images and containers not removed
- 8 Insufficient disk space on the node
- 9 Evicted Pods due to DiskPressure
- 10 Garbage collection not working or misconfigured

WHERE EPHEMERAL STORAGE IS USED



INVESTIGATION STEPS

- 1 Check node conditions and disk usage
- 2 Check inode usage
- 3 Identify which directories are using space
- 4 Check container logs size
- 5 Check container images and layers
- 6 Inspect emptyDir volumes
- 7 Check ephemeral-storage requests and limits
- 8 Look for large temporary files or caches
- 9 Review recent changes or deployments
- 10 Clean up safely and monitor

USEFUL COMMANDS

kubectl describe node <node>	Check node conditions
kubectl top node	View node resource usage
df -h	Check disk space usage
df -i	Check inode usage
du -sh /var/log/containers	Check container logs size
du -sh /var/lib/kubelet	Check kubelet storage usage
crictl images	List container images
crictl ps -a	List containers (containerd)
kubectl get pods -A -o wide	Check Pod storage usage
kubectl describe pod <pod>	Check emptyDir volumes
kubectl get pods -A -o yaml	Check storage requests/limits
crictl rmi <image-id>	Remove unused images
journalctl -u kubelet -f	Check eviction and GC logs

CHECK NODE DISK AND INODES

```
# Check disk space
df -h
Filesystem      Size  Used Avail Use%
/dev/sda1       100G   92G   8G   92%

# Check inode usage
df -i
Filesystem      Inodes IUsed IFree IUse%
/dev/sda1        6.5M   6.4M  120K  98%
```

FIND LARGE DIRECTORIES

```
# Find largest directories
du -h --max-depth=1 | sort -hr
12G    /var
8.1G   /var/lib
4.3G   /var/log
2.1G   /tmp
```

EPHEMERAL STORAGE RESOURCES

```
apiVersion: v1
kind: Pod
metadata:
  name: my-app
spec:
  containers:
  - name: app
    image: nginx
    resources:
      requests:
        ephemeral-storage: 100Mi
      limits:
        ephemeral-storage: 1Gi
```

Key Points

- ✓ Set appropriate ephemeral-storage requests and limits.
- ✓ Monitor usage.
- ✓ Prevent unbounded log and temporary file growth.
- ✓ Use log rotation.
- ✓ Clean up unused images and containers.

COMMON SIGNS AND EVENTS

EVENT / MESSAGE	MEANING
DiskPressure	Node is low on disk space or inodes
Pod evicted	Pod removed due to DiskPressure
Failed to create pod sandbox	Not enough ephemeral storage
no space left on device	Disk is full
too many open files	Inode exhaustion
ImageGC failed	Image garbage collection issue
Eviction thresholds exceeded	Kubelet triggered evictions

FIXES AND BEST PRACTICES

- ✓ Clean up old container images and unused containers.
- ✓ Enable and configure log rotation.
- ✓ Set proper ephemeral-storage requests and limits.
- ✓ Monitor disk and inode usage.
- ✓ Use size limits for emptyDir (if possible).
- ✓ Review application logging and temporary file usage.
- ✓ Configure kubelet eviction thresholds.
- ✓ Regularly run image and container garbage collection.
- ✓ Implement monitoring and alerts.

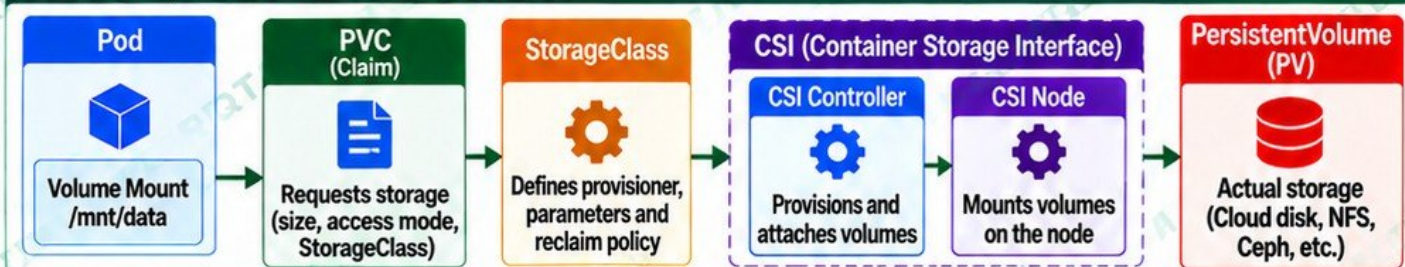
WHAT IT MEANS

- A PersistentVolume (PV) provides storage in the cluster.
- A PersistentVolumeClaim (PVC) requests storage from a StorageClass.
- Failures can keep PVCs in Pending or cause attach/mount errors.
- Often caused by StorageClass issues, CSI driver problems, topology mismatches, access mode conflicts or underlying infrastructure issues.
- Can lead to application downtime or potential data loss if not handled properly.

COMMON CAUSES

- 1 PVC stuck in Pending (no matching PV or StorageClass)
- 2 Wrong or missing StorageClass
- 3 Unsupported or incorrect access modes
- 4 CSI controller or node driver not running
- 5 Volume attach or mount failures
- 6 Multi-attach errors (volume already in use)
- 7 Topology or zone/region mismatch
- 8 Insufficient storage quota or capacity
- 9 Network or API connectivity issues to the storage backend
- 10 Permission, authentication or IAM issues
- 11 Underlying cloud/infra storage failures
- 12 Misconfiguration or incompatible CSI driver version

HOW PV, PVC AND CSI WORKS



INVESTIGATION STEPS

- 1 Check PVC status, events and description
- 2 Verify StorageClass and parameters
- 3 Check if a matching PV exists
- 4 Review access modes and size
- 5 Check CSI controller and node pods
- 6 Check attach/mount events and errors
- 7 Verify topology, zones and node constraints
- 8 Look for multi-attach conflicts
- 9 Check storage backend health and quotas
- 10 Review network, IAM and permissions
- 11 Identify the root cause and decide on fix
- 12 Consider data safety and backup before changes

USEFUL COMMANDS

<code>kubectl get pvc -A</code>	List all PVCs
<code>kubectl describe pvc <name></code>	View PVC details and events
<code>kubectl get pv</code>	List all PVs
<code>kubectl describe pv <name></code>	View PV details
<code>kubectl get storageclass</code>	List StorageClasses
<code>kubectl describe sc <name></code>	View StorageClass config
<code>kubectl get pods -n kube-system</code>	Check CSI controller/node pods
<code>kubectl logs <csi-pod></code>	View CSI driver logs
<code>kubectl get events --sort-by=.lastTimestamp</code>	Check recent events
<code>kubectl get nodes -o wide</code>	Check node topology and zones
<code>kubectl describe node <name></code>	Check node conditions and mounts
<code>kubectl get csidrivers</code>	List installed CSI drivers
<code>kubectl get csinodes</code>	List CSI node registrations

CHECK PVC STATUS

```
$ kubectl get pvc
```

NAME	STATUS	VOLUME	CAPACITY
data	Bound	pv-123	10Gi
logs	Pending	<none>	<none>
cache	Bound	pv-456	20Gi

CHECK EVENTS (EXAMPLE)

```
$ kubectl describe pvc logs
```

Events:

Type	Reason	Message
Warning	ProvisioningFailed	failed to provision with StorageClass "fast-ssd"

COMMON ERROR MESSAGES

ERROR MESSAGE	LIKELY CAUSE
PVC pending	No matching StorageClass or insufficient capacity
Failed to attach volume	CSI or cloud storage issue
Failed to mount volume	Node, CSI or filesystem issue
Multi-Attach error	Volume already in use by another node
MountVolume.SetUp failed	Permission, path or driver issue
No space left on device	Disk full on node or volume
Volume not found	Wrong volume ID or deleted PV

DATA-SAFETY DECISIONS

- ✓ Identify if the volume contains critical data.
- ✓ Take a backup or snapshot before making changes.
- ✓ Understand the reclaim policy (Retain, Delete).
- ✓ Avoid force-deleting PVs without confirming data.
- ✓ For stateful applications, follow a safe migration plan.
- ✓ Use ReadWriteOnce volumes on a single node unless multi-node access is supported.
- ✓ Verify data integrity after recovery.
- ✓ Work with your storage provider for underlying issues.

KEY TAKEAWAYS

- ✓ PVC Pending usually means StorageClass, capacity or configuration issues.
- ✓ Check events for exact error messages.
- ✓ Ensure correct access modes and size.
- ✓ Keep CSI controller and node drivers healthy.
- ✓ Verify topology and zones.
- ✓ Watch for multi-attach errors.
- ✓ Monitor storage capacity and quotas.
- ✓ Always consider data safety before making changes.
- ✓ Have a backup and recovery plan.

Deployment Rollout Is Failing

ReplicaSets, rollout status, readiness, maxSurge, maxUnavailable, progress deadlines, failed releases, rollback versus roll-forward.

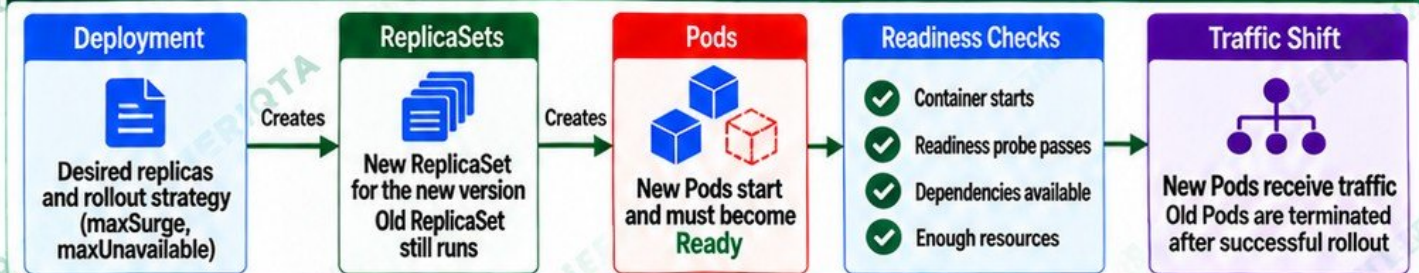
WHAT IT MEANS

- A Deployment rollout is not completing successfully.
- New Pods are not becoming ready or are failing during rollout.
- The desired number of replicas is not reached within the progress deadline.
- Caused by application errors, misconfiguration, resource issues, readiness probe failures or image problems.
- Can leave the application in a partial or unavailable state if not resolved.

COMMON CAUSES

- 1 Pods failing to start (crash, ImagePullBackOff, etc.)
- 2 Readiness probe failing
- 3 Application errors or bad configuration
- 4 Insufficient resources (CPU, memory)
- 5 Wrong image tag or image not found
- 6 maxSurge and maxUnavailable misconfiguration
- 7 Progress deadline exceeded
- 8 ReplicaSet not scaling correctly
- 9 Dependency service or database not available
- 10 Network, DNS or storage issues
- 11 Failed migration or schema changes
- 12 Security, permission or policy blocking the Pods

HOW A DEPLOYMENT ROLLOUT WORKS



INVESTIGATION STEPS

- 1 Check Deployment status and rollout events
- 2 Check ReplicaSets and Pods
- 3 Review Pod logs and describe output
- 4 Check readiness probe and application health
- 5 Verify image, config, Secrets and environment variables
- 6 Check resource requests, limits and node resources
- 7 Review maxSurge, maxUnavailable and strategy
- 8 Check progress deadline and rollout history
- 9 Investigate dependencies (services, databases, etc.)
- 10 Check network, DNS and storage connectivity
- 11 Decide on fix: rollback or roll-forward
- 12 Verify successful rollout and application health

USEFUL COMMANDS

<code>kubectl get deployments</code>	List all Deployments
<code>kubectl describe deployment <name></code>	View details and events
<code>kubectl get rs</code>	List ReplicaSets
<code>kubectl get pods -l app=<name></code>	List Pods for the Deployment
<code>kubectl describe pod <pod></code>	Check Pod details and events
<code>kubectl logs <pod></code>	View application logs
<code>kubectl rollout status deployment <name></code>	Check rollout progress
<code>kubectl rollout history deployment <name></code>	View rollout history
<code>kubectl rollout undo deployment <name></code>	Rollback to previous version
<code>kubectl set image deployment <name> \</code> <code>container=<container>=<image></code>	Roll forward with new image
<code>kubectl get events</code> <code>--sort-by=.lastTimestamp</code>	View recent events
<code>kubectl get pods -o wide</code>	Check node and IP details

CHECK ROLLOUT STATUS

```
$ kubectl rollout status deployment/my-app
Waiting for deployment "my-app"
to rollout...
1 out of 3 new replicas have
been updated...
3 out of 3 new replicas have
been updated.
deployment "my-app" successfully
rolled out
```

CHECK ROLLOUT HISTORY

```
$ kubectl rollout history deployment/my-app
```

REVISION	CHANGE-CAUSE
1	Initial version
2	Update image to v2
3	Update config
4	Update image to v3

DEPLOYMENT STRATEGY SETTINGS

SETTING	DESCRIPTION
maxSurge	Number of extra Pods allowed above desired replicas during rollout
maxUnavailable	Number of Pods allowed to be unavailable during rollout
progressDeadlineSeconds	Time to wait for rollout to complete before it is considered failed
strategy.type	RollingUpdate (default) or Recreate

ROLLBACK vs ROLL-FORWARD

ROLLBACK (REVERT)	ROLL-FORWARD (FIX AND CONTINUE)
✓ Use when new version is broken	✓ Fix the issue (image, config, resources, etc.)
✓ Quickly return to previous working version	✓ Deploy a new version
✓ Use <code>kubectl rollout undo</code>	✓ Use <code>kubectl set image</code> or update the manifest
✓ Minimal downtime	✓ Verify rollout and application health

KEY TAKEAWAYS

- ✓ Monitor rollout status and events.
- ✓ Ensure readiness probes are correct.
- ✓ Set appropriate maxSurge and maxUnavailable values.
- ✓ Use progress deadline to detect failures.
- ✓ Check logs, resources and dependencies.
- ✓ Use rollback for quick recovery.
- ✓ Use roll-forward after fixing the issue.
- ✓ Always verify application health after rollout.

Ingress Works Until It Doesn't

Ingress/Gateway path, controller health, Service backend, TLS, certificates, DNS, load balancers, health checks, 4xx versus 5xx isolation.

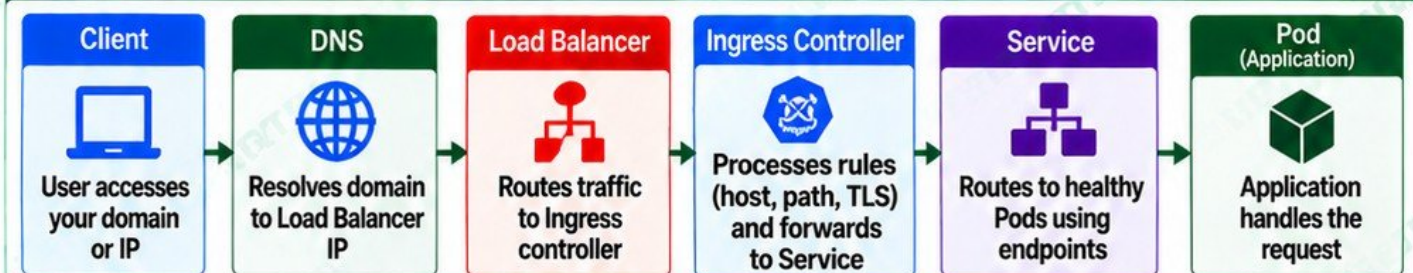
WHAT IT MEANS

- External traffic reaches the cluster but the application is not accessible or not working correctly.
- Ingress or Gateway resource exists but traffic is not routed to the correct backend Service.
- Can be caused by controller issues, DNS, load balancer, TLS/certificate problems, Service or Pod issues.
- Symptoms include timeouts, 4xx errors (client side) or 5xx errors (server side).

COMMON CAUSES

- 1 Ingress controller not running or unhealthy
- 2 Wrong or missing Ingress/Gateway configuration
- 3 Service backend not found or no healthy endpoints
- 4 DNS not pointing to the correct load balancer
- 5 TLS or certificate issues (expired, incorrect, not trusted)
- 6 Load balancer not configured correctly
- 7 Health check failures
- 8 Path, host or rewrite rule misconfiguration
- 9 4xx errors due to client, auth or routing issues
- 10 5xx errors due to backend application issues
- 11 Network policy, firewall or security group blocking traffic
- 12 Resource limits or high load on the ingress controller

HOW INGRESS TRAFFIC FLOWS



INVESTIGATION STEPS

- 1 Check Ingress/Gateway resource and annotations
- 2 Verify ingress controller pods and logs
- 3 Check Service and EndpointSlices
- 4 Test DNS resolution and load balancer
- 5 Verify TLS certificate and expiration
- 6 Check load balancer health checks
- 7 Test application directly (bypass ingress)
- 8 Identify 4xx vs 5xx errors
- 9 Review ingress controller logs and metrics
- 10 Check network policies and firewall rules
- 11 Verify path, host and rewrite rules
- 12 Fix the issue and confirm end-to-end access

USEFUL COMMANDS

<code>kubectl get ingress</code>	List Ingress resources
<code>kubectl describe ingress <name></code>	View details and events
<code>kubectl get gateway</code>	List Gateway resources
<code>kubectl describe gateway <name></code>	View Gateway details
<code>kubectl get pods -n <ns></code>	Check ingress controller pods
<code>kubectl logs <pod> -n <ns></code>	View controller logs
<code>kubectl get svc <name></code>	Check Service details
<code>kubectl get endpoints <name></code>	View endpoints
<code>kubectl get endpointslices</code>	List EndpointSlices
<code>kubectl describe endpointslice <name></code>	View endpoint details
<code>kubectl get cert -A</code>	List certificates (cert-manager)
<code>kubectl describe cert <name></code>	View certificate status
<code>nslookup <domain></code>	Test DNS resolution
<code>curl -I https://<domain></code>	Test external access (headers)
<code>curl -v https://<domain></code>	Verbose request for troubleshooting

CHECK INGRESS CONTROLLER

```
$ kubectl get pods -n ingress-nginx
```

NAME	READY	STATUS	RESTARTS
ingress-nginx-7d8f9	1/1	Running	0
ingress-nginx-5f6d7	1/1	Running	0

CHECK INGRESS RESOURCE

```
$ kubectl describe ingress my-app
```

```

Name:      my-app
Class:     nginx
Host:      app.example.com
Address:   1.2.3.4
TLS:      app-tls (Ready)
Rules:    / -> my-service:80
Events:   <none>
  
```

COMMON HTTP ERRORS

STATUS	MEANING
200	Request successful
301/302	Redirect (check rules and TLS)
400	Bad request (client issue)
401	Unauthorized (auth issue)
403	Forbidden (permission issue)
404	Not found (wrong path/host)
500	Internal server error (backend issue)
502	Bad gateway (controller cannot reach service)
503	Service unavailable (no healthy endpoints)
504	Gateway timeout (backend slow or unreachable)

TLS AND CERTIFICATE CHECKS

```
$ kubectl get cert -A
```

NAME	READY	SECRET	AGE
app-tls	True	app-tls-secret	15d

```
$ kubectl describe cert app-tls
```

```

Status:      Ready
Not After:   2026-12-31T23:59:59Z
Issuer:      letsencrypt-prod
Events:     <none>
  
```

KEY TAKEAWAYS

- ✓ Always verify DNS and load balancer first.
- ✓ Check ingress controller health and logs.
- ✓ Ensure Service has healthy endpoints.
- ✓ Validate TLS certificate and renewal.
- ✓ Test application directly inside the cluster.
- ✓ Use logs and metrics to identify 4xx vs 5xx.
- ✓ Review ingress rules, annotations and paths.
- ✓ Check network policies and firewall rules.
- ✓ Monitor controller, load balancer and backend.
- ✓ Have a rollback plan for configuration changes.

NetworkPolicy and CNI Failures

Pod networking, CNI health, allowed flows, default-deny mistakes, node-to-Pod and Pod-to-Pod failures, packet-path debugging.

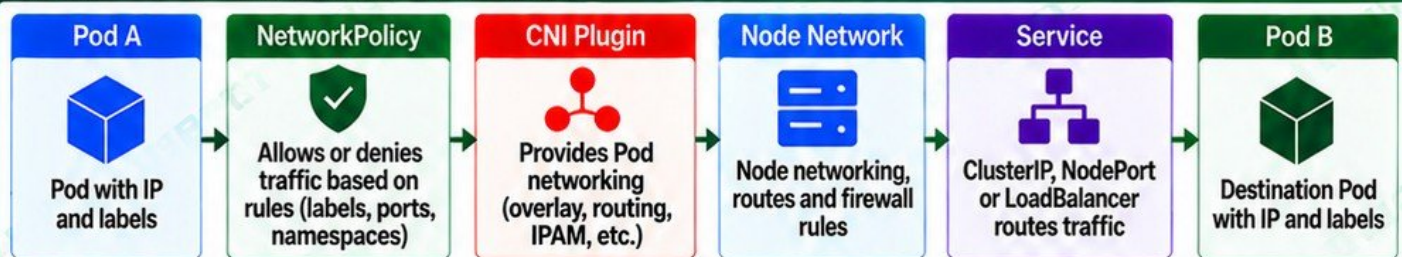
WHAT IT MEANS

- Kubernetes networking allows Pods, Services and cluster components to communicate with each other.
- NetworkPolicy controls which traffic is allowed or denied between Pods, namespaces and external sources.
- CNI (Container Network Interface) provides the network connectivity for Pods.
- Failures can be caused by NetworkPolicy rules, CNI plugin issues, node networking problems, or misconfigured rules.
- Symptoms include no connectivity between Pods, node-to-Pod failures or external access issues.

COMMON CAUSES

- 1 NetworkPolicy default-deny blocking traffic
- 2 Incorrect or missing NetworkPolicy rules
- 3 CNI plugin not running or unhealthy
- 4 CNI misconfiguration
- 5 Node networking issues (routes, firewall, IP forwarding)
- 6 Pod-to-Pod communication blocked
- 7 Node-to-Pod communication blocked
- 8 DNS traffic blocked by NetworkPolicy
- 9 Service or ingress traffic blocked
- 10 Wrong namespaces or label selectors
- 11 Calico, Cilium, Flannel or other CNI specific issues
- 12 MTU, overlay network or packet drop issues

HOW POD NETWORKING WORKS



INVESTIGATION STEPS

- 1 Check Pod and node network connectivity
- 2 Verify NetworkPolicy rules and selectors
- 3 Check CNI plugin pods and logs
- 4 Test DNS connectivity
- 5 Check Service and EndpointSlices
- 6 Verify node routes and firewall rules
- 7 Test Pod-to-Pod, node-to-Pod and external access
- 8 Capture and analyze network packets
- 9 Check for default-deny policies
- 10 Verify namespaces and labels
- 11 Review CNI configuration and health
- 12 Identify the root cause and fix the issue

USEFUL COMMANDS

<code>kubectl get pods -n kube-system</code>	Check CNI plugin pods
<code>kubectl logs -n kube-system <pod></code>	View CNI logs
<code>kubectl get networkpolicy -A</code>	List all NetworkPolicies
<code>kubectl describe networkpolicy <name></code>	View policy details
<code>kubectl get pod -o wide</code>	Check Pod IP and node
<code>kubectl get nodes -o wide</code>	Check node network info
<code>kubectl get svc -A</code>	List Services
<code>kubectl get endpointslices -A</code>	Check service endpoints
<code>kubectl exec -it <pod> -- nslookup <name></code>	Test DNS from Pod
<code>kubectl exec -it <pod> -- ping <ip></code>	Test connectivity
<code>kubectl exec -it <pod> -- curl -v <url></code>	Test HTTP/HTTPS access
<code>tcpdump -i any host <ip></code>	Capture packets on node
<code>ip route show</code>	Check node routes
<code>iptables -L -n</code>	Check firewall rules (node)

CHECK CNI HEALTH

```
$ kubectl get pods -n kube-system
```

NAME	READY	STATUS	RESTARTS
calico-node-abc	1/1	Running	0
coredns-xyz	1/1	Running	0
kube-proxy-def	1/1	Running	0

CHECK NETWORKPOLICY

```
$ kubectl get networkpolicy -n prod
```

NAME	POD-SELECTOR	POLICY-TYPES
deny-all	app=web	Ingress
allow-dns	app=web	Ingress, Egress

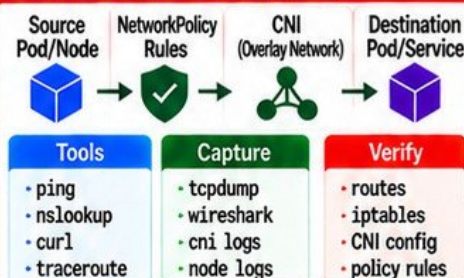
TEST CONNECTIVITY FROM POD

```
$ kubectl exec -it web-pod -- sh
/ # nslookup api.default.svc
/ # ping 10.96.0.1
/ # curl -v http://api.default.svc
/ # curl -v https://google.com
```

COMMON NETWORKING ISSUES

ISSUE	COMMON CAUSE
Pod-to-Pod fails	NetworkPolicy blocking traffic
Node-to-Pod fails	Node firewall or routing issue
DNS not working	NetworkPolicy blocking DNS (UDP 53)
Service unreachable	Wrong selectors or missing endpoints
External access fails	CNI or node network misconfiguration
Intermittent drops	MTU, overlay or network congestion
All Pods no network	CNI plugin down or misconfigured

PACKET-PATH DEBUGGING



KEY TAKEAWAYS

- ✓ Understand how your CNI provides Pod networking.
- ✓ Use NetworkPolicy to allow only required traffic.
- ✓ Always check CNI plugin health and logs.
- ✓ Test connectivity at each step of the packet path.
- ✓ Differentiate between DNS, network, policy and service issues.
- ✓ Use packet capture to confirm the root cause.
- ✓ Monitor network policies and CNI components.
- ✓ Avoid accidental default-deny mistakes.

RBAC: Forbidden, Unauthorized and Broken Access

Authentication versus authorization, ServiceAccounts, Roles/ClusterRoles, bindings, impersonation checks, least privilege, audit evidence.

WHAT IT MEANS

- RBAC controls who can do what in the cluster.
- Authentication verifies your identity (who you are).
- Authorization (RBAC) determines your permissions (what you can do).
- Access can be denied due to missing or incorrect Roles, ClusterRoles or bindings.
- ServiceAccounts are used by Pods and applications to access the API.
- Symptoms include "Forbidden" or "Unauthorized" errors when running kubectl or from applications.

COMMON CAUSES

- 1 Missing or incorrect Role or ClusterRole
- 2 Missing or incorrect RoleBinding or ClusterRoleBinding
- 3 User or ServiceAccount has no permissions
- 4 Wrong namespace in Role or binding
- 5 Authentication works but authorization fails
- 6 Using the wrong context or cluster
- 7 ServiceAccount token issues
- 8 Overly restrictive or misconfigured RBAC rules
- 9 Impersonation not allowed
- 10 API group, resource or verb not permitted
- 11 ClusterRoleBinding used when only namespace access is needed
- 12 No audit logs to verify the denied request

HOW KUBERNETES RBAC WORKS



INVESTIGATION STEPS

- 1 Check the exact error message
- 2 Verify your current user and context
- 3 Check if you are authenticated
- 4 Use auth can-i to test permissions
- 5 Review ServiceAccount and token (for Pods)
- 6 Check Role, ClusterRole and bindings
- 7 Verify the correct namespace
- 8 Check API group, resource and verb
- 9 Test using impersonation (if allowed)
- 10 Review audit logs for denied requests
- 11 Identify the missing or incorrect RBAC rule
- 12 Fix the issue with least privilege access

USEFUL COMMANDS

<code>kubectl config get-contexts</code>	List available contexts
<code>kubectl config current-context</code>	Show current context
<code>kubectl auth whoami</code>	Show current user (if supported)
<code>kubectl auth can-i <verb> <resource></code>	Check if you have permission
<code>kubectl auth can-i -n <ns> <verb> <resource></code>	Check permission in a namespace
<code>kubectl get serviceaccounts -n <ns></code>	List ServiceAccounts
<code>kubectl get roles -n <ns></code>	List Roles
<code>kubectl get clusterroles</code>	List ClusterRoles
<code>kubectl get rolebindings -n <ns></code>	List RoleBindings
<code>kubectl get clusterrolebindings</code>	List ClusterRoleBindings
<code>kubectl describe role <name> -n <ns></code>	View Role details
<code>kubectl describe clusterrole <name></code>	View ClusterRole details
<code>kubectl describe rolebinding <name> -n <ns></code>	View RoleBinding details
<code>kubectl describe clusterrolebinding <name></code>	View ClusterRoleBinding details
<code>kubectl auth can-i --as=<user> <verb> <resource></code>	Test using user impersonation
<code>kubectl auth can-i --as=system:serviceaccount-<ns>:<sa> <verb> <resource></code>	Test using ServiceAccount impersonation

COMMON ERROR MESSAGES

ERROR MESSAGE	MEANING
Forbidden	You are authenticated but not authorized.
Unauthorized	Authentication failed or invalid credentials.
cannot <verb> <resource>	Missing RBAC permission for the action.
User "X" cannot <verb> <resource>	The user does not have the required role/binding.
ServiceAccount not found	Incorrect or missing ServiceAccount.
no matches for kind	Wrong API group or resource name.

CHECK RBAC FOR A POD

```

$ kubectl get pod <pod> -n <ns> -o yaml | grep serviceAccountName
$ kubectl describe serviceaccount <sa> -n <ns>
$ kubectl get role,rolebinding -n <ns>
$ kubectl get clusterrole,clusterrolebinding
  
```

RBAC OBJECTS OVERVIEW

OBJECT	PURPOSE
ServiceAccount	Identity for Pods and applications
Role	Permissions within a namespace
ClusterRole	Permissions across the cluster
RoleBinding	Binds a Role to a user or ServiceAccount
ClusterRoleBinding	Binds a ClusterRole to a user or ServiceAccount

BEST PRACTICES

- ✓ Use least privilege access.
- ✓ Create namespace scoped Roles when possible.
- ✓ Use ClusterRoles only when required.
- ✓ Avoid using cluster-admin for applications.
- ✓ Regularly review and audit RBAC permissions.
- ✓ Use ServiceAccounts for Pods instead of user credentials.
- ✓ Monitor audit logs for denied requests.
- ✓ Use impersonation to test permissions before applying.
- ✓ Remove unused Roles, ClusterRoles and bindings.

KEY TAKEAWAYS

- ✓ Authentication and authorization are different.
- ✓ RBAC controls what you can do in Kubernetes.
- ✓ ServiceAccounts are used by Pods.
- ✓ Check the exact error message.
- ✓ Use `kubectl auth can-i` to test permissions.
- ✓ Verify Roles, ClusterRoles and bindings.
- ✓ Apply least privilege access.
- ✓ Use audit logs for evidence and troubleshooting.
- ✓ Regularly review and clean up RBAC rules.

API Server Slow, Unavailable or Overloaded

API latency/errors, admission webhooks, client storms, watches, throttling, etcd dependency, control-plane saturation and blast radius.

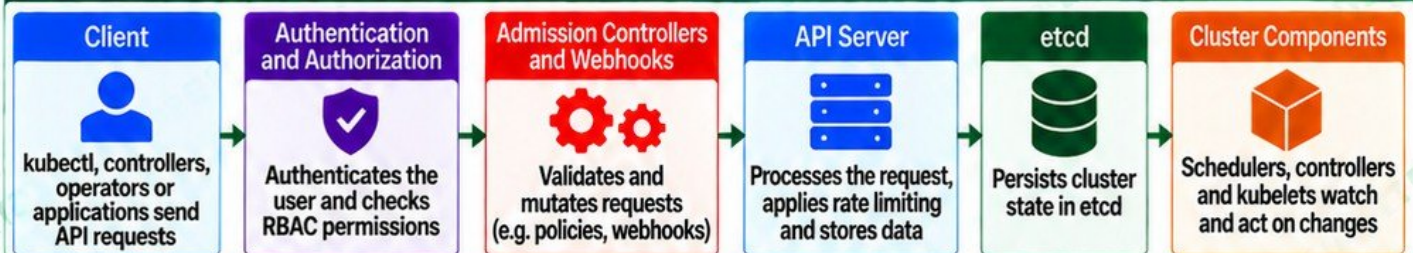
WHAT IT MEANS

- The Kubernetes API server is the central control plane component that receives and processes all requests.
- It can become slow, return errors or be unavailable due to high load, misconfigurations or dependency issues.
- Issues can be caused by client request storms, watch activity, admission webhooks, etcd problems or control-plane resource saturation.
- When the API server is affected, the entire cluster can be impacted because all components and users depend on it.
- Symptoms include high latency, timeouts, 429 (Too Many Requests), 5xx errors or complete unavailability.

COMMON CAUSES

- High number of API requests (client storm)
- Excessive watch connections
- Slow or failing admission webhooks
- API rate limiting and throttling (429 errors)
- etcd latency or unavailability
- Large objects (ConfigMaps, Secrets, CRDs)
- Control-plane CPU or memory saturation
- Network issues between control-plane components
- Misconfigured or slow API aggregation layer
- Expensive API queries (wide list requests)
- Too many controllers or operators
- DDoS or external traffic to the API server

HOW THE API SERVER WORKS



INVESTIGATION STEPS

- Check API server health and endpoint
- Check API server logs
- Verify API latency and error rates
- Identify client request storms
- Check admission webhooks
- Check etcd health and latency
- Check control-plane resource usage
- Look for throttling and 429 errors
- Check watch connections and open requests
- Review network connectivity and firewall rules
- Identify large or expensive API requests
- Determine blast radius and impacted services

USEFUL COMMANDS

<code>kubectl get --raw=/healthz</code>	Check API server health
<code>kubectl get --raw=/readyz</code>	Check readiness
<code>kubectl get --raw=/livez</code>	Check liveness
<code>kubectl get --raw=/metrics</code>	View API server metrics
<code>kubectl logs -n kube-system \</code> <code>-l component=kube-apiserver</code>	View API server logs
<code>kubectl get apiservices</code>	Check API aggregation layer
<code>kubectl get validatingwebhookconfigurations</code>	List validating webhooks
<code>kubectl get mutatingwebhookconfigurations</code>	List mutating webhooks
<code>kubectl get --raw=/api/v1/namespaces/</code> <code>kube-system/endpoints/kubernetes'</code>	Check API endpoint
<code>kubectl get events -n kube-system</code>	View related events
<code>kubectl top pod -n kube-system</code>	Check control-plane resource usage
<code>etcdctl endpoint status</code>	Check etcd endpoint health

CHECK API SERVER METRICS

<code>kubectl get --raw=/metrics grep apiserver_</code>	
<code>apiserver_request_total</code>	Total requests
<code>apiserver_request_duration_seconds</code>	Request latency
<code>apiserver_current_inflight_requests</code>	Current inflight
<code>apiserver_admission_controller_admission_duration_seconds</code>	Webhook latency
<code>apiserver_flowcontrol_rejected_requests_total</code>	Throttled requests
<code>apiserver_watch_events_total</code>	Watch activity
<code>apiserver_storage_objects</code>	Objects in storage

COMMON ERROR MESSAGES

ERROR MESSAGE	MEANING
429 Too Many Requests	Client is being throttled
500 Internal Server Error	API server or etcd issue
503 Service Unavailable	API server is overloaded or down
context deadline exceeded	Request timed out
i/o timeout	Network or etcd latency
connection refused	API server not reachable
etcdserver: request timed out	etcd is slow or unavailable

CONTROL-PLANE DEPENDENCIES

COMPONENT	DEPENDENCY ON API SERVER
etcd	Stores all cluster state
kube-scheduler	Watches API for Pods and nodes
kube-controller-manager	Watches API for cluster resources
kubelet	Reports node and Pod status to API
kubect / users	Send all requests to API server
Admission webhooks	Called by API server during requests
Operators / controllers	Continuously watch the API

ROOT CAUSE AREAS

Client Storms - Too many requests - Misconfigured apps - Large listings	Admission Webhooks - Slow webhooks - External service issues - Timeouts
etcd Issues - High latency - Disk or network issues - Leader election problems	Resource Saturation - High CPU or memory - Too many watches - Control-plane overload

KEY TAKEAWAYS

- Monitor API server latency, errors and resource usage. Identify and fix client request storms.
- Ensure admission webhooks are fast and reliable. Keep etcd healthy and highly available.
- Avoid large or unnecessary API requests. Watch for throttling and 429 errors.
- Review audit logs for evidence.
- Understand the blast radius of API server issues. Implement proper rate limiting and client retries.
- Regularly test control-plane components.

etcd and Control Plane Survival

etcd health, quorum, latency, disk I/O, database growth, leader changes, snapshots, restore risk, why casual intervention can destroy the cluster.

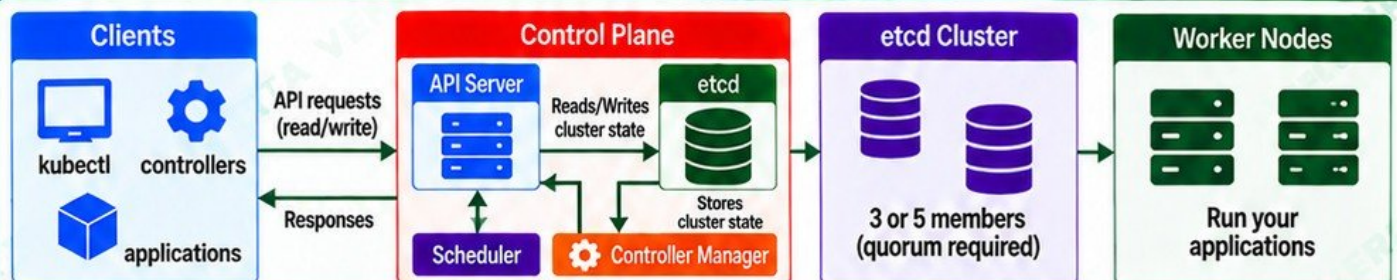
WHAT IT MEANS

- etcd is the distributed database that stores all cluster state.
- The control plane (API server, etcd, scheduler, controller manager) keeps the cluster running.
- etcd must have a quorum (majority) to operate.
- If etcd is unhealthy, the API server becomes unavailable and the entire cluster is affected.
- Common issues include high latency, disk I/O problems, database growth, leader changes and loss of quorum.
- etcd is critical. Casual or incorrect intervention can corrupt data and destroy the cluster.

COMMON CAUSES

- Low disk space or high disk I/O latency
- Large database size due to many objects or old data
- Loss of quorum (majority of etcd members unavailable)
- Network issues between etcd members
- High latency or slow disk performance
- Leader election changes or instability
- API server overload or client request storms
- Misconfigured etcd settings (e.g. election timeouts)
- Too many events, logs or large objects (ConfigMaps, Secrets)
- Failed or incomplete snapshots
- Incorrect or unsafe manual changes to etcd
- Node failure or control-plane component failure

HOW etcd FITS IN THE CONTROL PLANE



INVESTIGATION STEPS

- Check etcd member health and status
- Verify cluster quorum
- Check etcd logs for errors and warnings
- Review disk space and disk I/O performance
- Check etcd database size and growth rate
- Look for leader election changes
- Check API server health and latency
- Identify client request storms or high load
- Review network connectivity between etcd members
- Verify snapshot schedule and last successful snapshot
- Check control-plane component health
- Identify root cause and plan safe remediation

USEFUL COMMANDS

<code>kubectl get componentstatuses</code>	Check control-plane health
<code>kubectl get --raw=/healthz</code>	Check API server health
<code>kubectl get --raw=/readyz</code>	Check API server readiness
<code>etcdctl member list</code>	List etcd members
<code>etcdctl endpoint status</code>	Check member health and latency
<code>etcdctl endpoint health</code>	Check endpoint health
<code>etcdctl member list -w table</code>	View members in table format
<code>etcdctl alarm list</code>	Check for alarms
<code>etcdctl endpoint status -w table</code>	View detailed status
<code>df -h</code>	Check disk space
<code>iostat -x 1 5</code>	Check disk I/O performance
<code>du -sh /var/lib/etcd</code>	Check etcd data size
<code>kubectl logs -n kube-system <etcd-pod></code>	View etcd logs
<code>kubectl get events -n kube-system</code>	Check cluster events

CHECK etcd MEMBER STATUS

```
$ etcdctl member list -w table
```

ID	STATUS	NAME	PEER URL	CLIENT URL
a1b2c3	started	etcd-1	https://10.0.0.1	https://10.0.0.1
d4e5f6	started	etcd-2	https://10.0.0.2	https://10.0.0.2
g7h8i9	started	etcd-3	https://10.0.0.3	https://10.0.0.3

CHECK etcd ENDPOINT HEALTH

```
$ etcdctl endpoint status -w table
```

ENDPOINT	ID	VERSION	DB SIZE	IS LEADER	RTT
10.0.0.1	a1b2c3	3.5.10	1.2 GB	yes	5ms
10.0.0.2	d4e5f6	3.5.10	1.1 GB	no	6ms
10.0.0.3	g7h8i9	3.5.10	1.3 GB	no	7ms

CHECK DISK SPACE AND I/O

```
$ df -h /var/lib/etcd
```

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/sda1	100G	70G	30G	70%	/var/lib/etcd

```
$ iostat -x 1 3
```

Device	r/s	w/s	rMB/s	wMB/s	await	%util
sda	120	350	12.5	25.3	45.2	80.5

etcd HEALTH INDICATORS

METRIC	HEALTHY	WARNING	CRITICAL
Member Status	started	slow/unstable	unreachable
Quorum	majority up	degraded	no quorum
Latency (RTT)	< 10 ms	10 - 50 ms	> 50 ms
DB Size Growth	stable	growing fast	uncontrolled
Disk Space	> 20% free	10 - 20% free	< 10% free
Disk I/O Latency	low	high	very high
Leader Changes	infrequent	occasional	frequent
API Server	healthy	slow responses	unavailable

SNAPSHOT AND RESTORE

Take a Snapshot	Restore a Snapshot
- Use regular automated snapshots - Store snapshots safely (off-cluster) - Verify snapshot integrity - Keep multiple versions	- Use only when necessary - Follow official documentation - Restore to a clean environment - Be aware of data loss risk - Validate cluster after restore
<pre># Take a snapshot etcdctl snapshot save /backup/etcd-snap-\$(date +%F).db</pre> <pre># Verify snapshot etcdctl snapshot status /backup/etcd-snap-2026-01-01.db</pre>	<pre># Restore from snapshot (example) etcdctl snapshot restore /backup/etcd-snap-2026-01-01.db \ --data-dir /var/lib/etcd-new</pre>

KEY TAKEAWAYS

- etcd is critical for cluster survival.
- Maintain quorum at all times.
- Monitor latency, disk I/O and database size.
- Take regular snapshots and verify them.
- Avoid large numbers of unnecessary objects.
- Do not make casual or manual changes to etcd.
- Investigate leader changes and instability.
- Ensure control-plane components are healthy.
- Understand the blast radius of etcd issues.
- Follow official Kubernetes documentation for any recovery actions.

Cluster Capacity and Scaling Failures

HPA, VPA, Cluster Autoscaler, requests, quotas, scheduler capacity, pending workloads, cloud quotas, scale-up delays, cost versus headroom.

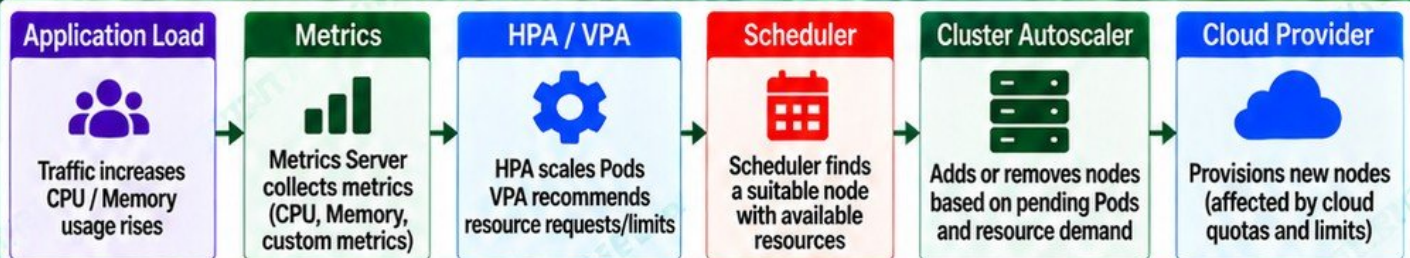
WHAT IT MEANS

- Cluster capacity is the available compute resources (CPU, memory, storage and nodes) to run your workloads.
- Scaling components like HPA, VPA and Cluster Autoscaler adjust resources and nodes based on demand.
- Failures happen when there are not enough resources, wrong requests/limits, quotas, scheduler constraints or cloud provider limits.
- Symptoms include pending Pods, slow scale-up, application latency, throttling and unexpected costs.
- Proper capacity planning balances performance, reliability and cost.

COMMON CAUSES

- HPA misconfigured or wrong metrics
- VPA in the wrong mode or causing restarts
- Cluster Autoscaler not running or misconfigured
- CPU or memory requests too high or too low
- Resource quotas or limit ranges blocking workloads
- Not enough node capacity in the cluster
- Cloud provider instance or regional quota limits
- Slow node provisioning (image download, networking, etc.)
- Scheduler cannot find a node due to taints, affinity or resources
- Large number of pending Pods
- Scale-up delays due to cloud API limits
- Over-provisioning (high cost) or under-provisioning (insufficient headroom)

HOW SCALING AND CAPACITY WORKS



INVESTIGATION STEPS

- Check Pod status and identify pending Pods
- Describe the Pod to see scheduler events
- Check node resource usage and capacity
- Verify HPA, VPA and Cluster Autoscaler configuration
- Check resource requests and limits
- Review resource quotas and limit ranges
- Check cloud provider quotas and instance limits
- Look at scheduler events and failure reasons
- Check node scale-up events and timings
- Verify metrics availability (Metrics Server)
- Analyze cost versus required headroom
- Identify the root cause and apply the right fix

USEFUL COMMANDS

<code>kubectl get pods</code>	View Pod status
<code>kubectl describe pod <name></code>	Check scheduling events
<code>kubectl get nodes -o wide</code>	View node capacity and usage
<code>kubectl top nodes</code>	Check node resource usage
<code>kubectl top pods</code>	Check Pod resource usage
<code>kubectl get hpa</code>	List HPA resources
<code>kubectl describe hpa <name></code>	View HPA details
<code>kubectl get vpa</code>	List VPA resources
<code>kubectl describe vpa <name></code>	View VPA details
<code>kubectl get quota</code>	List resource quotas
<code>kubectl describe quota</code>	View quota details
<code>kubectl get limitrange</code>	List limit ranges
<code>kubectl describe limitrange</code>	View limit range details
<code>kubectl get events --sort-by='lastTimestamp'</code>	Check recent cluster events
<code>kubectl get clusterautoscaler -n kube-system</code>	Check Cluster Autoscaler
<code>kubectl logs -n kube-system -l app=cluster-autoscaler</code>	View Cluster Autoscaler logs

CHECK POD SCHEDULING EVENTS

```
$ kubectl describe pod my-app-xyz
```

Type	Reason	Message
Warning	FailedScheduling	0/3 nodes are available: 3 Insufficient cpu.
Warning	FailedScheduling	0/3 nodes are available: 3 Insufficient memory.
Normal	TriggeredScaleUp	pod triggered scale-up: 1 node(s) were added to the cluster.

CHECK CLUSTER AUTOSCALER LOGS

```
$ kubectl logs -n kube-system \
-l app=cluster-autoscaler
```

I0915 12:00:01	Scale-up: setting node group size to
I0915 12:00:45	Successfully added 1 node(s) the cluster

COMMON SCALING ISSUES

ISSUE	COMMON CAUSE
Pending Pods	Not enough nodes or high requests
HPA not scaling	Wrong metrics or thresholds
VPA causing restarts	VPA in Auto mode with incompatible apps
Cluster Autoscaler not scaling	Misconfiguration or cloud quota limits
Slow node provisioning	Image downloads, network setup, cloud API delays
Quota exceeded	ResourceQuota or cloud provider limit
Scheduler cannot find node	Taints, affinity, insufficient resources
Unexpected high costs	Over-provisioning or no limits
Application latency	Insufficient resources or delayed scaling

SCALING COMPONENTS OVERVIEW

COMPONENT	PURPOSE
HPA (Horizontal Pod Autoscaler)	Automatically scales the number of Pods based on metrics (CPU, memory or custom metrics)
VPA (Vertical Pod Autoscaler)	Recommends or automatically adjusts resource requests/limits for Pods
Cluster Autoscaler	Automatically adds or removes nodes based on pending Pods and resource demand
Resource Quotas	Limits total resource usage in a namespace
Limit Ranges	Sets default and maximum resource requests/limits

KEY TAKEAWAYS

- Monitor resource usage and set realistic requests and limits.
- Use HPA for traffic-based scaling and VPA for right-sizing resources.
- Ensure Cluster Autoscaler is correctly configured and has permission to scale nodes.
- Check and manage resource quotas and cloud provider limits.
- Plan for headroom to handle traffic spikes.
- Balance cost and performance.
- Regularly review scaling events and metrics.
- Fix the root cause, not just the symptom.

The Production Cluster Emergency Playbook

Full incident workflow covering triage, stabilization, evidence preservation, change freeze, mitigation, recovery, verification, communication and prevention.



WHAT IT MEANS

- A structured step-by-step process to handle critical incidents in a production Kubernetes cluster.
- Helps you respond quickly, minimize impact and restore services safely.
- Covers the full lifecycle from initial alert to long-term prevention.
- Ensures you preserve evidence, communicate effectively and make the right decisions.
- Reduces chaos, mistakes and further damage during incidents.

COMMON INCIDENTS

- 1 Cluster or API server unavailable
- 2 Widespread Pod failures
- 3 Node failures or resource exhaustion
- 4 Storage or PersistentVolume issues
- 5 Network and DNS outages
- 6 Ingress or external access down
- 7 etcd issues
- 8 High latency or application errors
- 9 Misconfigurations or bad deployments
- 10 Security incident or unauthorized access
- 11 Cloud provider or underlying infrastructure issues
- 12 Multiple component failures (major outage)

THE INCIDENT WORKFLOW



DETAILED STEPS

- 1 Triage: confirm the incident and assess impact
- 2 Stabilization: stop further damage
- 3 Evidence: collect logs, metrics and events
- 4 Change freeze: pause non-essential changes
- 5 Mitigation: apply temporary fixes
- 6 Recovery: restore services
- 7 Verification: confirm everything is healthy
- 8 Communication: keep stakeholders updated
- 9 Prevention: identify root cause and improve
- 10 Document: write a full incident report
- 11 Review: conduct a post-incident review
- 12 Follow up: implement action items and monitor

USEFUL COMMANDS

<code>kubectl get nodes -o wide</code>	Check node status
<code>kubectl get pods -o wide</code>	Check all Pods
<code>kubectl get events -A --sort-by=.lastTimestamp</code>	View recent events
<code>kubectl get svc,ingress -A</code>	Check services and ingress
<code>kubectl get pv,pvc -A</code>	Check storage
<code>kubectl get hpa,vpa -A</code>	Check autoscalers
<code>kubectl top nodes</code>	Check node resource usage
<code>kubectl top pods -A</code>	Check Pod resource usage
<code>kubectl describe <resource></code>	Detailed resource information
<code>kubectl logs <pod> -n <ns></code>	View Pod logs
<code>kubectl get componentstatuses</code>	Check control-plane health
<code>kubectl get --raw='/readyz?verbose'</code>	Check API server readiness
<code>kubectl config view</code>	Check current context
<code>etcdctl endpoint status</code>	Check etcd health (if access)
<code>etcdctl member list</code>	List etcd members

EVIDENCE TO COLLECT

- ✓ `kubectl get events -A`
- ✓ `kubectl describe` key resources
- ✓ Pod logs and previous logs
- ✓ Control-plane and etcd logs
- ✓ Metrics from monitoring system
- ✓ API server logs and audit logs
- ✓ Network and DNS information
- ✓ Recent changes (deployments, configs)
- ✓ Cloud provider events and health
- ✓ Screenshots and timestamps

COMMUNICATION TEMPLATE

- What is happening (short summary)
- Impact (which services are affected)
- Current status (what is being done)
- Next update time
- Resolution summary (after recovery)

INCIDENT SEVERITY LEVELS

SEVERITY	EXAMPLE
SEV-1 (Critical)	Cluster down, major outage
SEV-2 (High)	Multiple services affected
SEV-3 (Medium)	Partial service degradation
SEV-4 (Low)	Minor issue, no major impact

KEY ROLES DURING INCIDENT

ROLE	RESPONSIBILITY
Incident Commander	Leads the response and decisions
Technical Team	Investigates and fixes the issue
Communications	Updates stakeholders
Scribe	Documents timeline and actions
Approver	Authorizes major changes

KEY TAKEAWAYS

- ✓ Stay calm and follow a structured process.
- ✓ Preserve evidence before making changes.
- ✓ Stop non-essential changes.
- ✓ Communicate clearly and regularly.
- ✓ Verify full recovery before closing.
- ✓ Learn from the incident and implement long-term improvements.
- ✓ Regularly test your incident response plan.